

JAVA PROGRAMMING LABORATORY

Experiment No: 05

Date: 21/09/2026

Title: Demonstration of ListIterator Interface Methods in Java

1. Objective / Aim

To understand and demonstrate the ListIterator interface in Java for forward and backward traversal of a list and for modifying list elements using next(), previous(), nextIndex(), previousIndex(), set(), add(), and remove().

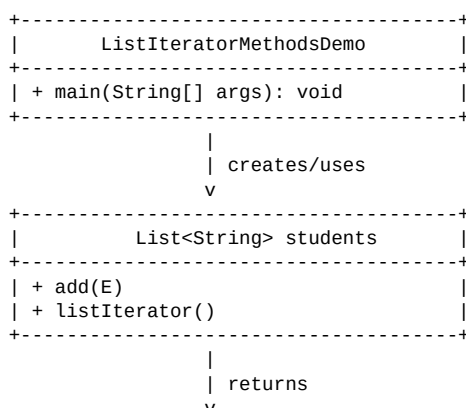
2. System Requirements

Hardware Requirements	Software Requirements
Computer/Laptop	Operating System: Windows / Linux / macOS
Minimum 4 GB RAM	Java Development Kit (JDK) 17 or above
Minimum 1 GHz processor	IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
At least 100 MB free storage	Java compiler and Command Prompt/Terminal

3. Algorithm / Logic Description

1. Import ArrayList, List, and ListIterator from java.util.
2. Create a List of String elements using ArrayList.
3. Add four student names to the list.
4. Display the original list.
5. Create a ListIterator using listIterator().
6. Use hasNext(), next(), and nextIndex() to traverse the list in the forward direction.
7. Use hasPrevious(), previous(), and previousIndex() to traverse the list backward.
8. Create a new ListIterator positioned at the beginning of the list.
9. Use next() to move to the first element and set() to replace Rahul with Aarav.
10. Use add() to insert Diya at the current iterator position.
11. Move to the next element using next() and use remove() to delete that element.
12. Display the list after each modification.

4. UML Class Diagram





5. Source Code

```

import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

class ListIteratorMethodsDemo {

    public static void main(String[] args) {

        // Creating a List
        List<String> students = new ArrayList<>();

        students.add("Rahul");
        students.add("Priya");
        students.add("Arjun");
        students.add("Meera");

        System.out.println("Original List: " + students);

        // Creating ListIterator
        ListIterator<String> iterator = students.listIterator();

        // hasNext(), next(), nextIndex()
        System.out.println("\nForward Traversal:");

        while (iterator.hasNext()) {

            System.out.println(
                "Index: " + iterator.nextIndex() +
                ", Student: " + iterator.next()
            );
        }

        // hasPrevious(), previous(), previousIndex()
        System.out.println("\nBackward Traversal:");

        while (iterator.hasPrevious()) {

            System.out.println(
                "Index: " + iterator.previousIndex() +
                ", Student: " + iterator.previous()
            );
        }

        // Move to first element
        iterator = students.listIterator();

        // next() and set()
        iterator.next();
        iterator.set("Aarav");

        System.out.println("\nAfter set(): " + students);

        // add()
        iterator.add("Diya");

        System.out.println("After add(): " + students);

        // Move to next element and remove it
    }
}

```

```
        iterator.next();
        iterator.remove();

        System.out.println("After remove(): " + students);
    }
}
```

6. Completion and Execution Commands

Step 1: Save the program as:
ListIteratorMethodsDemo.java

Step 2: Compile the program:
javac ListIteratorMethodsDemo.java

Step 3: Execute the program:
java ListIteratorMethodsDemo

7. Output

Original List: [Rahul, Priya, Arjun, Meera]

Forward Traversal:

Index: 0, Student: Rahul
Index: 1, Student: Priya
Index: 2, Student: Arjun
Index: 3, Student: Meera

Backward Traversal:

Index: 3, Student: Meera
Index: 2, Student: Arjun
Index: 1, Student: Priya
Index: 0, Student: Rahul

After set(): [Aarav, Priya, Arjun, Meera]
After add(): [Aarav, Diya, Priya, Arjun, Meera]
After remove(): [Aarav, Diya, Arjun, Meera]

8. Result / Completion

The program was successfully compiled and executed. The ListIterator interface methods were demonstrated for forward and backward traversal and for modifying the elements of an ArrayList using set(), add(), and remove().